

## Fundamentos del Análisis de Datos

### Tarea #1: Paquete Jerárquico para Análisis Estadístico

**Alumno:** Diego Moisés Rojas Mata  
**Profesor:** M.I. Juan José Cárdenas Cornejo  
**Fecha de entrega:** 20 de Septiembre de 2026

## 1. Introducción

El propósito fundamental de este proyecto es diseñar, estructurar e implementar un paquete jerárquico en Python orientado al análisis estadístico univariante de datos agrupados.

El sistema busca cumplir con el patrón de diseño de doble contexto (`__name__ == "__main__"`), lo que permite que el paquete funcione como una librería importable reutilizable o como un script de auditoría automatizada. Los objetivos específicos incluyen la creación de módulos independientes para calcular distribuciones de frecuencias, medidas de tendencia central, medidas de dispersión y métricas de forma mediante un flujo de datos secuencial e interconectado.

## 2. Metodología

Para abordar el problema, se optó por un enfoque modular utilizando las estructuras de control básicas de Python (ciclos `for`, `while` y condicionales) y diccionarios para el tránsito de información, manteniendo el principio de simplicidad del lenguaje.

El flujo de procesamiento comienza en el script cliente `main.py`, el cual genera una serie sintética de 100 valores aleatorios. Estos datos crudos se envían al módulo `frecuencias.py`, donde se aplica la Regla de Sturges para calcular los intervalos, límites y frecuencias (absolutas, relativas y acumuladas). Posteriormente, el diccionario resultante se inyecta en cadena hacia los módulos de `tendencias.py`, `dispersion.py` y `forma.py`, reutilizando cálculos previos como la media aritmética o la desviación estándar para evitar redundancias.

A continuación, se presenta el diagrama a bloques que explica el funcionamiento de la implementación y la interrelación de los módulos:

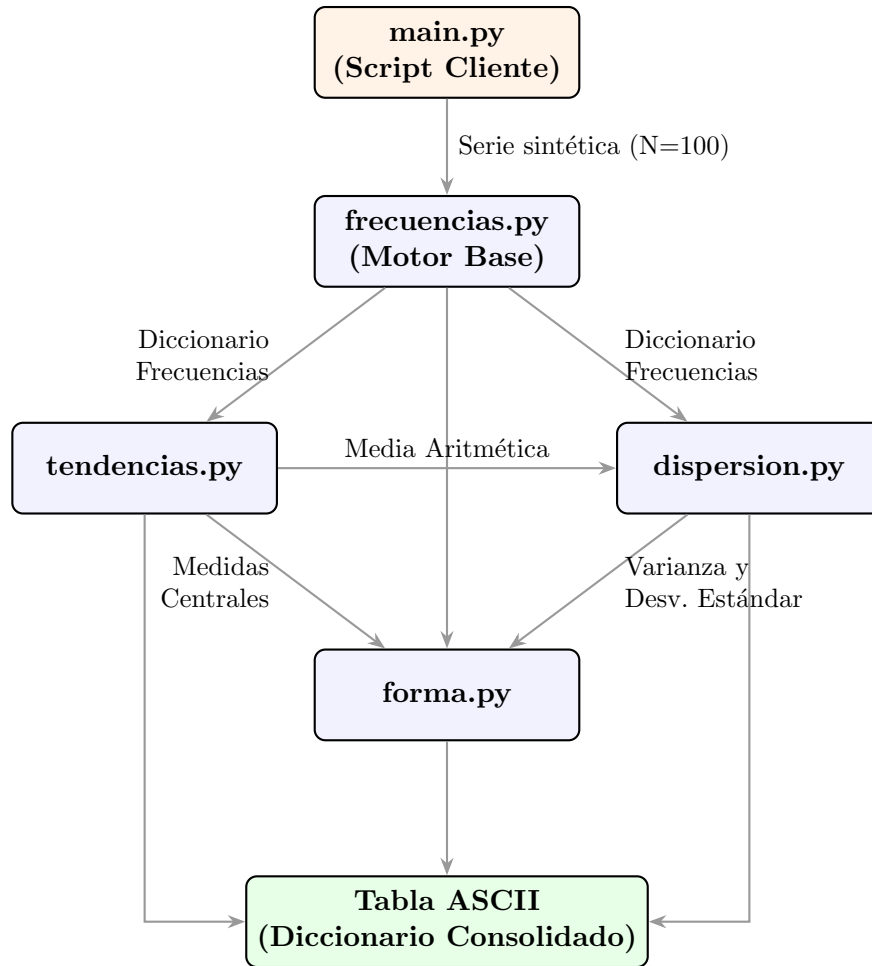


Figura 1: Diagrama a bloques ilustrando la interrelación modular y el flujo del pipeline de datos estadísticos.

### 3. Resultados

Tras ejecutar el script `main.py` en su contexto principal de auditoría, se procesó con éxito una serie sintética de valores. El sistema logró empaquetar todos los resultados de los cuatro submódulos en un diccionario consolidado.

A continuación, se muestra la salida de la terminal con el formato de tabla limpio y alineado mediante f-strings y caracteres de caja, tal como se especificó en los requerimientos:

```

Problems  Output  Debug Console  Terminal  Ports

- Sistema de Análisis Estadístico -
Se han generado 100 datos continuos y 100 datos discretos

Elige el tipo de datos (1 = Continuos, 2 = Discretos): 1

- Resultados -
+-----+-----+
| Métrica | Resultado |
+-----+-----+
| Total Datos (N) | 100 |
| Total Clases | 8 |
| Amplitud | 10.9546 |
| Media Aritmetica | 56.7084 |
| Media Geometrica | 50.6843 |
| Media Armonica | 44.0467 |
| Mediana | 55.8321 |
| Moda | 73.3595 |
| RCM | 61.6011 |
| Cuartil 1 | 35.0183 |
| Cuartil 2 | 55.8321 |
| Cuartil 3 | 76.5241 |
| Decil 1 | 23.6985 |
| Decil 2 | 31.0016 |
| Decil 3 | 40.4956 |
| Decil 4 | 48.9854 |
| Decil 5 | 55.8321 |
| Decil 6 | 67.3953 |
| Decil 7 | 73.4812 |
| Decil 8 | 80.4800 |
| Decil 9 | 89.6918 |
| Percentil 10 | 23.6985 |
| Percentil 25 | 35.0183 |
| Percentil 50 | 55.8321 |
| Percentil 75 | 76.5241 |
| Percentil 90 | 89.6918 |
| Percentil 99 | 98.6547 |
| Rango | 87.6370 |
| Rango Semi-Intercuartilico | 20.7529 |
| Desviacion Media | 21.0329 |
| Varianza | 578.8509 |
| Desviacion Estandar | 24.0593 |
| Chebyshev (k=2) | 0.7500 |
| Coeficiente Variacion (%) | 42.4263 |
| Z-Score (Clase 1) | -1.6300 |
| Fisher-Pearson | -0.0480 |
| Asimetria Pearson 1 | -0.6921 |
| Asimetria Pearson 2 | 0.1093 |
| Sesgo Cuantiles | -0.0029 |
| Exceso Curtosis | -1.1962 |
| Curtosis Percentilica | 1.5900 |
+-----+-----+

```

Figura 2: Auditoría automatizada y visualización tabular de resultados en consola.

## 4. Análisis y Discusión

Los resultados obtenidos demuestran que las fórmulas teóricas se tradujeron correctamente a código Python. Un hallazgo importante durante el desarrollo fue la necesidad de mantener el aislamiento estricto de las variables; por ejemplo, al calcular la Raíz Cuadrada Media (RCM) o las varianzas, fue vital iterar individualmente sobre cada índice de las listas de frecuencias y marcas de clase para evitar errores de sintaxis al operar colecciones enteras.

Asimismo, se comprobó que empaquetar los cálculos en diccionarios fue la estructura de datos más eficiente para la interrelación de los módulos, ya que permitió a funciones

complejas (como el Z-Score o el Exceso de Curtosis) acceder inmediatamente a parámetros previamente calculados como la desviación estándar o la media aritmética, reduciendo el costo computacional.

## 5. Conclusiones

El desarrollo de esta primera tarea me demostró que la programación orientada al análisis de datos va mucho más allá de transcribir fórmulas matemáticas; se trata de estructurar la lógica de manera ordenada. Al construir este paquete estadístico desde cero, logré afianzar mi dominio sobre las estructuras de control básicas (ciclos `for`, `while` y condicionales) y descubrí la enorme utilidad de los diccionarios para hacer que la información fluya sin enredos entre distintos archivos.

Uno de los mayores retos fue comprender el aislamiento de variables al operar con listas. Por ejemplo, al programar la varianza o la Raíz Cuadrada Media (RCM), fue crucial aprender a iterar sobre cada índice de las frecuencias y marcas de clase individualmente, en lugar de intentar aplicar operaciones algebraicas directamente a las colecciones enteras.

Finalmente, ver la auditoría automatizada ejecutarse sin errores en el script `main.py` bajo el patrón de doble contexto, me hizo comprobar que el principio de “simple es mejor que complejo” realmente funciona en la práctica. Me quedo con la satisfacción de haber creado un paquete estadístico que no solo cumple con los resultados esperados, sino que cuenta con una arquitectura de código limpia, modular y construida con mi propio estilo.